MEMORIA RISC-V

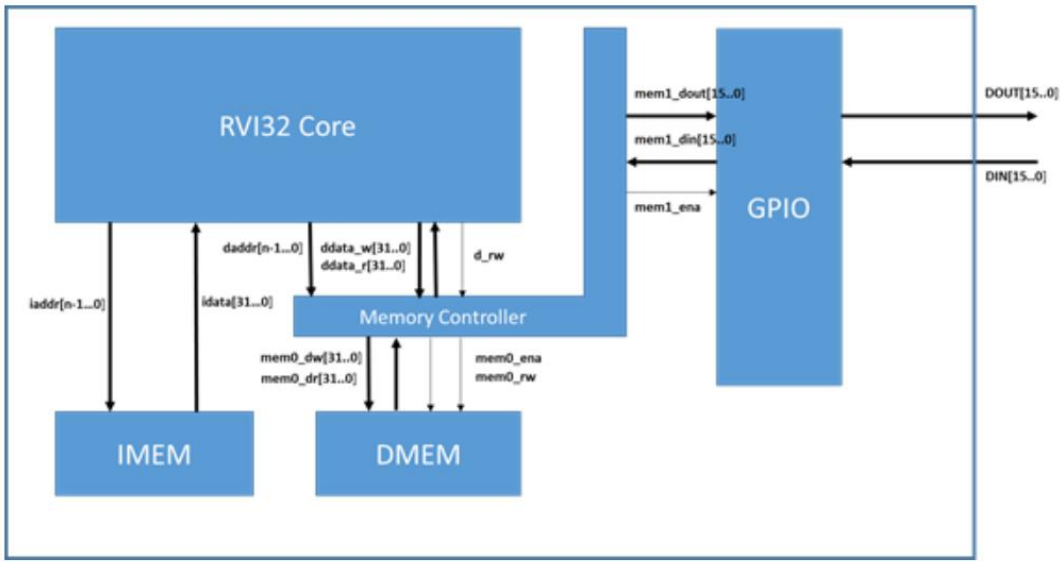
3PARS1

Jaime Piqueras Marsal, Gonzalo López de Villalta Hernández, Marc Zaragoza García, Mónica Blazquez Femenía, Paula García Badenas, Jorge López Martorell

1 INTRODUCCIÓN

El proyecto desarrollado tiene como objetivo la implementación, verificación funcional y validación experimental de un microcontrolador sencillo denominado TinuC, basado en un subconjunto del conjunto de instrucciones RISC-V. El procesador, de arquitectura RV32I, se describe íntegramente en SystemVerilog, garantizando su sintetizabilidad para su posterior implementación en una FPGA Cyclone IV de intelFPGA. Asimismo, se contempla el desarrollo de programas en lenguaje ensamblador destinados tanto a la evaluación del funcionamiento del procesador como a su uso en la etapa final de validación experimental en laboratorio.

El microcontrolador TinuC incorpora 31 registros de propósito general de 32 bits (x1–x31), un registro x0 cableado permanentemente a cero, y un contador de programa (pc) encargado de dirigir el flujo de ejecución. El diseño soporta los seis formatos de instrucción definidos en RISC-V (R, I, S, SB, U y UJ), todos ellos de 32 bits, e implementa un conjunto mínimo de instrucciones necesarias para la realización de operaciones aritmético-lógicas, comparaciones, control de flujo y acceso a memoria. Adicionalmente, se prevé la posibilidad de ampliar el soporte con instrucciones adicionales de salto y comparación.



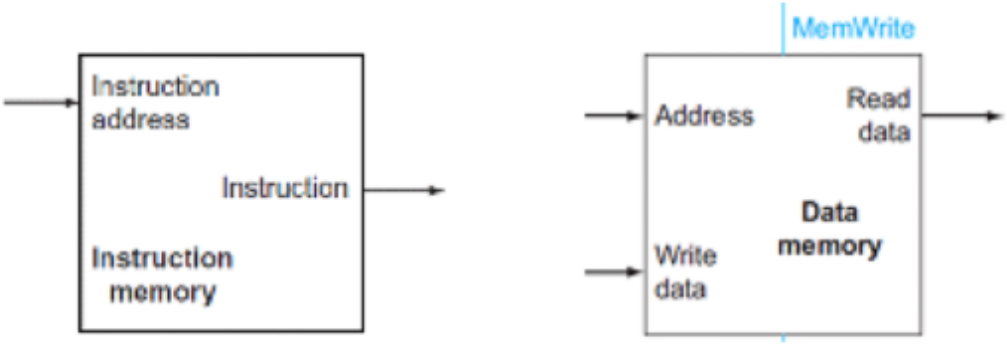
El proyecto se divide en cuatro fases: en la Fase 1, se diseña y valida el microprocesador con sus unidades funcionales (memoria, registros y ALU) usando programación RISC-V. La Fase 2 implementa un modelo single-cycle del procesador, validando su funcionamiento con programas sencillos. En la Fase 3, el procesador se segmenta en un pipeline de cinco etapas para mejorar el rendimiento, incorporando mecanismos de control de riesgos de datos. Finalmente, la Fase 4 lleva el microcontrolador a hardware mediante FPGA, validando su funcionamiento con un programa simple en la plataforma DE2-115. Cada fase incluye pruebas y validaciones antes de avanzar a la siguiente.

2 FASE 1: UNIDADES FUNCIONALES BÁSICAS

En la Fase 1 se desarrollan y verifican las unidades funcionales básicas del microcontrolador: la memoria de instrucciones (ROM), la memoria de datos (RAM), la ALU y el banco de registros.

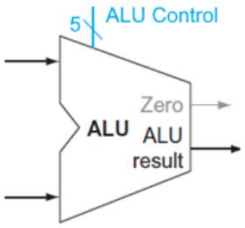
Memoria de instrucciones y datos:

La **ROM** debe tener 1024 posiciones de 32 bits y realizar lecturas asíncronas. La **RAM** implementa lecturas asíncronas y escrituras síncronas controladas por MemWrite. Ambas pueden inicializarse mediante las funciones $readmemh y $readmemb. Estos requisitos permiten ejecutar cada instrucción en un único ciclo dentro del modelo single-cycle.



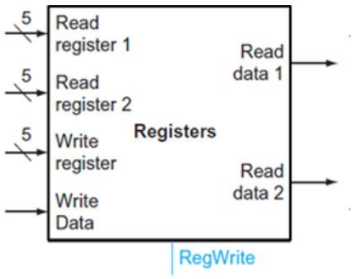
Unidad aritmético-lógica (ALU):

La **ALU**, de carácter combinacional, opera con dos operandos de 32 bits y utiliza un campo de 4 bits para seleccionar la operación correspondiente. Incluye además un flag zero para soportar instrucciones de salto condicional.



Banco de registros:

El **banco de registros** almacena los 32 registros del procesador y debe permitir dos lecturas asíncronas y una escritura síncrona habilitada por RegWrite, acorde con la estructura de las instrucciones RISC-V.



La finalidad de esta fase es garantizar que todos estos módulos funcionan correctamente de manera individual, estableciendo la base necesaria para la posterior integración en el procesador single-cycle de la Fase 2.

Adicionalmente, en esta fase se diseña la **Unidad de Control**, la cual es responsable de generar las señales de control necesarias para coordinar las operaciones entre la ALU, las memorias y los registros. La unidad de control decodifica las instrucciones y controla el flujo de datos entre las distintas unidades del procesador.

También se define el modelo de programación **RISC-V**, especificando las instrucciones que el procesador debe ser capaz de ejecutar, incluyendo instrucciones aritméticas, de control de flujo y de acceso a memoria. Este conjunto de instrucciones permitirá que el procesador ejecute operaciones de cálculo y control.

Finalmente, durante esta fase se lleva a cabo la validación funcional de cada unidad. Se realizan pruebas sencillas, como la ejecución de programas en ensamblador RISC-V, para asegurarse de que las unidades funcionales trabajen correctamente antes de avanzar a las siguientes fases del proyecto.

2.1 VERIFICACION FASE 1

En la verificación de la Fase 1, se lleva a cabo una validación funcional de las unidades diseñadas, asegurando que cada componente del microprocesador, como la ALU, las memorias y el banco de registros, funcione correctamente antes de pasar a fases más avanzadas del diseño. Para esta verificación, se deben realizar dos programas sencillos utilizando el simulador Venus:

Serie de Fibonacci: El primer programa consiste en calcular y almacenar en la memoria de datos los primeros N números de la sucesión de Fibonacci. Este valor N se define como una constante almacenada en memoria estática. Este programa validará que el microprocesador sea capaz de ejecutar correctamente una secuencia de operaciones aritméticas y almacenar los resultados en la memoria, comprobando la funcionalidad de las operaciones de carga y almacenamiento en la memoria de datos.

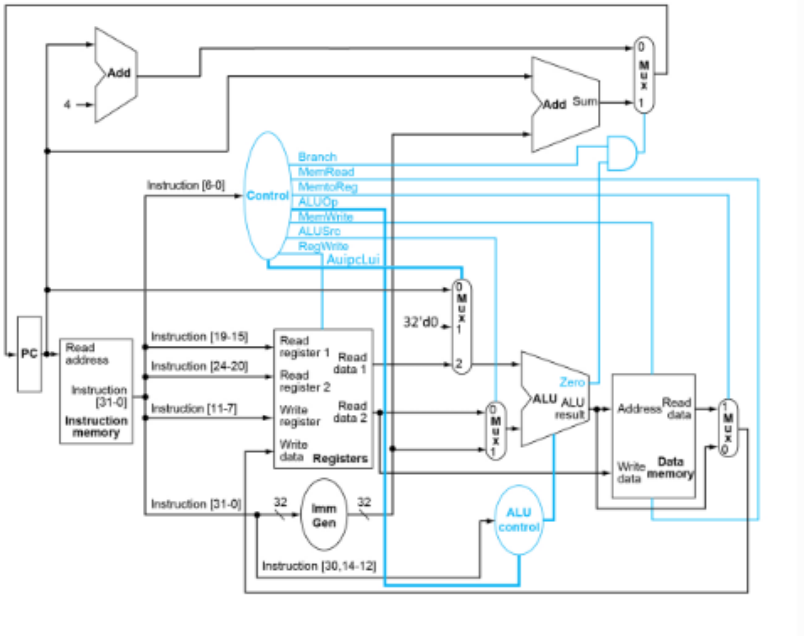
Algoritmo de Ordenamiento por Selección: El segundo programa consiste en ordenar una serie de números almacenados en la memoria estática utilizando el algoritmo de ordenamiento por selección. El microprocesador debe ser capaz de ordenar los datos

almacenados en la memoria de datos correctamente, lo que valida tanto las operaciones de acceso a memoria como la capacidad del procesador para ejecutar correctamente un algoritmo sencillo. En este caso, también se valida que los datos ordenados sean almacenados adecuadamente en la memoria de datos.

La verificación a través de estos programas permite asegurar que las unidades básicas del procesador (ALU, memoria y registros) operen correctamente bajo condiciones de prueba controladas antes de avanzar con el diseño y la implementación más complejos.

3 FASE 2: DISEÑO DEL PROCESADOR *SINGLE-CYCLE*

En la Fase 2 del proyecto, se procede con el diseño e implementación del procesador en un modelo **single-cycle**. Esta fase tiene como objetivo principal verificar que el procesador funcione correctamente ejecutando instrucciones de manera secuencial en un solo ciclo de reloj. Para ello, se valida la correcta interacción entre las diferentes unidades del procesador, como la ALU, los registros y las memorias.



Además de la implementación básica, se llevan a cabo pruebas exhaustivas con programas sencillos, como los de la Fase 1, para garantizar que cada componente funcione de acuerdo a lo esperado. El proceso de verificación también incluye la creación de un testbench estructurado que permita realizar pruebas automatizadas de todas las instrucciones implementadas. Este banco de pruebas valida la correcta ejecución de instrucciones en diferentes formatos, como R-format, I-format, S-format, B-format y U-format.

Una vez que el procesador sea validado en esta fase y se haya comprobado que todas las instrucciones se ejecutan correctamente, se podrá avanzar hacia las siguientes fases, donde se buscará mejorar el rendimiento y añadir características más complejas, como la

segmentación en pipeline y la integración con hardware físico. La Fase 2, por lo tanto, establece la base funcional del procesador antes de proceder con las optimizaciones y la implementación avanzada.

3.1 VERIFICACIÓN FASE 2

En la Fase 2, se procedió a la verificación funcional del procesador single-cycle desarrollado en la Fase 1, asegurando que todas las unidades del procesador trabajaran de manera conjunta y sin errores. Para ello, se utilizaron los programas Serie de Fibonacci y Algoritmo de Ordenamiento por Selección, desarrollados en la Fase 1, con el fin de validar las principales operaciones del procesador.

La verificación se realizó mediante el uso del simulador Venus, que permite cargar y ejecutar programas de ensamblador, así como comprobar la correcta ejecución tanto en código ensamblador como en código máquina. Esta herramienta fue utilizada para probar las instrucciones implementadas en el procesador, tales como instrucciones aritméticas (como ADD y SUB), instrucciones de control de flujo (como BEQ y JAL) y instrucciones de acceso a memoria (como LW y SW).

Para facilitar la verificación y asegurar que el procesador ejecutara correctamente todas las instrucciones, se creó un testbench automatizado. Este testbench ejecutó las instrucciones de manera aleatoria y comprobó los resultados de cada una, validando el funcionamiento de la ALU, el banco de registros y las memorias de instrucciones y datos.

Cada instrucción fue vinculada a su respectivo programa de prueba, con una tabla detallada que especificaba las líneas de código que verificaban la correcta ejecución de las instrucciones. Esto permitió realizar una comprobación exhaustiva y sistemática del procesador single-cycle, asegurando que cada componente estuviera funcionando de acuerdo con lo esperado.

4 FASE 3: IMPLEMENTACIÓN SEGMENTADA *(PIPELINED)*

En la Fase 3, el procesador desarrollado en la fase anterior, single-cycle, se transforma en un procesador segmentado (**pipeline**). Este modelo divide la ejecución de las instrucciones en cinco etapas consecutivas, permitiendo la ejecución simultánea de varias instrucciones y, por tanto, mejorando el rendimiento global del procesador. Sin embargo, este enfoque introduce nuevos desafíos relacionados con los riesgos de datos, carga y control que deben gestionarse adecuadamente.

4.1 3. PIPELINE

La segmentación del procesador se realiza mediante la división del ciclo de ejecución de una instrucción en cinco etapas:

- IF (Instruction Fetch): Se obtiene la instrucción desde la memoria de instrucciones.
- ID (Instruction Decode): Se decodifica la instrucción y se lee el banco de registros.

- EX (Execute): Se ejecuta la operación aritmética o lógica, o se calcula la dirección de memoria.
- MEM (Memory Access): Se accede a la memoria de datos para leer o escribir operandos.
- WB (Write Back): Los resultados de la operación se escriben en el banco de registros.

El procesador segmentado mejora el rendimiento al permitir que diferentes instrucciones se encuentren en diferentes etapas del pipeline al mismo tiempo. Sin embargo, para que esta ejecución paralela sea eficiente, es necesario gestionar correctamente la interacción entre las etapas y la sincronización de los datos a través de las señales de control.

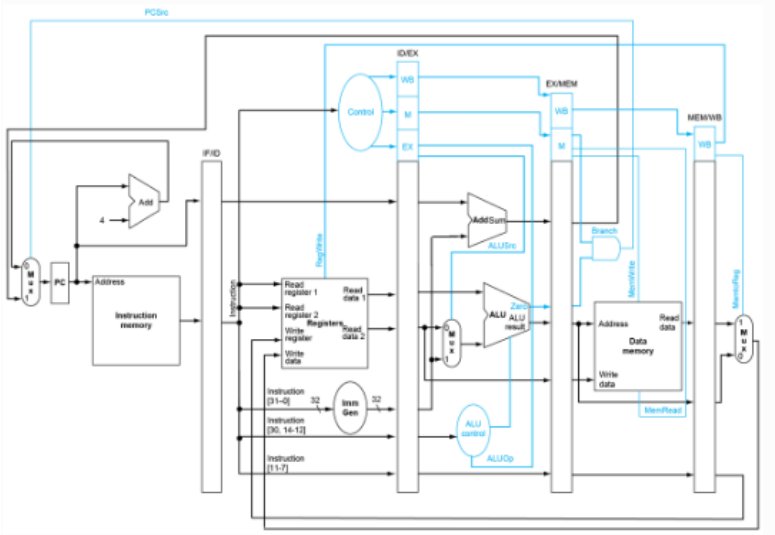


Fig. 7. Datapath (en negro) y controlpath (en azul) del procesador en versión segmentada.

4.2 3.1. AMPLIACIÓN DEL JUEGO DE INSTRUCCIONES

Durante esta sub-tarea, se amplió el conjunto de instrucciones para incluir nuevas operaciones de control de flujo y saltos. Las instrucciones añadidas incluyen:

JAL (Jump and Link) y JALR (Jump and Link Register), que permiten saltos incondicionales y el almacenamiento de la dirección de retorno.

BLT (Branch Less Than), BLTU (Branch Less Than Unsigned), BGE (Branch Greater Than or Equal), BGEU (Branch Greater Than or Equal Unsigned), que son instrucciones de salto condicional basadas en la comparación de registros.

Estas instrucciones requieren modificaciones tanto en la unidad de control como en el datapath para manejar los saltos y las condiciones de comparación de los registros. De esta manera, se asegura que el procesador puede realizar operaciones de control de flujo más avanzadas y gestionar adecuadamente el cambio de contexto durante la ejecución de las instrucciones.

En esta sub-tarea, aun no tenemos el microprocesador totalmente funcional, por lo que al hacer el TestBench debemos añadir algunos NOPs, por ejemplo, en el codigo de la serie de

fibonacci. De esta forma hace que espere un tiempo y dé un margen a todos los procesos que se ejecutan dentro.

En el siguiente apartado conseguiremos que el TestBench funcione sin los NOPs y de forma totalmente correcta, igual que cuando trabajábamos el modelo single-cycle.

4.3 3.2. RIESGOS EN EL PIPELINE

Con la segmentación del procesador, surgen nuevos riesgos que deben ser gestionados para asegurar el correcto funcionamiento del pipeline:

Riesgo de Datos: Se presenta cuando una instrucción depende de los resultados de una instrucción anterior que aún no ha finalizado. Este riesgo se puede gestionar mediante adelantamiento de datos (data forwarding), que permite que los resultados de una etapa del pipeline sean utilizados inmediatamente por las etapas posteriores sin esperar a que la instrucción anterior termine completamente. Este mecanismo mejora la eficiencia y reduce el tiempo de espera entre instrucciones.

En el diagrama de data forwarding, se observa cómo los muxes permiten seleccionar entre los datos provenientes de la memoria de datos o los resultados de la ALU para las entradas de la ALU, dependiendo de la etapa en la que se encuentre la instrucción. Esto ayuda a evitar que se detengan las instrucciones por esperar datos que ya están disponibles.

Riesgo de Carga: Este riesgo ocurre cuando una instrucción intenta utilizar un dato que está siendo cargado de la memoria en el mismo ciclo de ejecución. Para manejarlo, se introduce una burbuja en el pipeline, es decir, se inserta una NOP (instrucción nula) para retrasar la ejecución de la instrucción dependiente hasta que los datos sean completamente cargados.

Riesgo de Control: Los riesgos de control surgen en instrucciones de salto, como BEQ o JAL, donde el flujo de ejecución puede cambiar dependiendo de una condición. En estos casos, es necesario que el procesador detenga la ejecución de las instrucciones posteriores hasta que se resuelva la condición del salto. Las señales de control se desactivan temporalmente cuando se cumple una condición de salto, garantizando que las instrucciones siguientes no se ejecuten incorrectamente.

Tras añadir todas estas mejoras al código de la fase anterior ya conseguimos que el microprocesador trabaje de forma correcta ante cualquier situación. Mejorando así su rendimiento respecto al modelo single-cycle.

4.4 3.3. VERIFICACIÓN DEL PROCESADOR SEGMENTADO

Una vez implementado el procesador segmentado, es fundamental realizar una verificación exhaustiva para asegurar que todos los riesgos se gestionen adecuadamente y que las instrucciones se ejecuten de manera correcta:

Sub-tarea 3.3.1: Se modifican los programas de prueba, como la Serie de Fibonacci y el Algoritmo de Ordenamiento por Selección, para que puedan ejecutarse en el procesador segmentado. Además, se añaden instrucciones NOP en las ubicaciones necesarias para gestionar los riesgos de control.

Sub-tarea 3.3.2: Se verifica el procesador segmentado utilizando estos programas modificados, comprobando si el procesador maneja correctamente los riesgos de datos, control y carga. Este paso es crucial para asegurar que la segmentación no introduce errores en la ejecución de las instrucciones.

Sub-tarea 3.3.3: Se desarrolla un testbench automatizado que simula la ejecución de las instrucciones y verifica que el procesador funcione correctamente con el pipeline. Este testbench ayuda a comprobar que todas las etapas del pipeline interactúan de forma adecuada y que las soluciones de adelantamiento de datos y gestión de riesgos funcionan correctamente.

Con la validación completa del procesador segmentado, se asegura que el diseño está listo para avanzar a la siguiente fase del proyecto, que implicará su implementación en hardware FPGA para pruebas en una plataforma real.

5 FASE 4: IMPLEMENTACIÓN Y VALIDACIÓN HARDWARE DE TINUC

La Fase 4 tiene como objetivo la implementación del procesador en hardware utilizando la plataforma FPGA DE2-115. En esta fase, se transfiere el diseño del procesador segmentado a un entorno físico, validando su funcionamiento mediante aplicaciones que interactúan con los periféricos conectados. A continuación, se describen las actividades clave de esta fase:

5.1 4.1. IMPLEMENTACIÓN DEL PROCESADOR EN LA FPGA

En esta sub-tarea, el procesador segmentado diseñado en la fase anterior se implementa en la FPGA DE2-115. Para ello, se realiza la sintetización del diseño, generando un archivo de bitstream que puede ser cargado en la FPGA. Se utiliza la herramienta Quartus para realizar este proceso y verificar que el diseño sea funcional antes de la implementación en la placa.

Durante la implementación, se utiliza un Memory Controller que gestiona el mapa de memoria. Este controlador organiza la memoria en dos bloques principales:

- Memoria de Instrucciones (IMEM): Almacena las instrucciones que el procesador ejecutará.
- Memoria de Datos (DMEM): Almacena los datos que serán utilizados durante la ejecución de las instrucciones.

5.2 4.2. Integración con Periféricos

Una parte fundamental de la Fase 4 es la conexión del microprocesador con los periféricos a través de la interfaz GPIO (General Purpose Input/Output). Los periféricos conectados incluyen:

- Display de 7 segmentos: Utilizado para mostrar resultados de cálculos o valores.
- Pulsadores e interruptores: Permiten la entrada de datos y la interacción con el usuario.
- LEDs: Usados para mostrar el estado del sistema.

El GPIO conecta el procesador con estos periféricos, permitiendo tanto la entrada como la salida de datos, y gestionando la comunicación entre el microprocesador y los dispositivos externos.

5.3 4.3. Ejecución de Aplicaciones en Hardware

Una vez que el procesador está implementado y los periféricos conectados, se realizan pruebas funcionales para validar su funcionamiento. Las aplicaciones ejecutadas en esta fase incluyen:

Serie de Fibonacci: El procesador debe ser capaz de calcular los términos de la serie de Fibonacci y mostrarlos en el display de 7 segmentos.

Algoritmo de Ordenamiento por Selección: El procesador debe ordenar una serie de valores almacenados y mostrarlos en el display de 7 segmentos, lo que valida la capacidad del procesador para manejar operaciones más complejas.

Estas aplicaciones permiten evaluar la capacidad del procesador para ejecutar instrucciones, interactuar con los periféricos y realizar tareas básicas de computación en un entorno físico.

5.4 4.4. Validación de la Implementación en Hardware

Una vez que las aplicaciones se ejecutan correctamente, se lleva a cabo una validación final para asegurar que el procesador implementado en hardware funcione correctamente. Los aspectos validados incluyen:

Ejecución de instrucciones: Se valida que el procesador ejecute correctamente las instrucciones de la Serie de Fibonacci y del algoritmo de ordenamiento por selección.

Interacción con periféricos: Se verifica que la comunicación entre el procesador y los periféricos (interruptores, pulsadores, LEDs y display de 7 segmentos) funcione correctamente.

Estabilidad del sistema: Se comprueba que el sistema sea estable y no haya fallos en la ejecución, incluso cuando se introducen múltiples entradas de datos simultáneamente.

Se puede ver el video del funcionamiento correcto en la placa aquí:
https://youtube.com/shorts/KemECi3JWW0?feature=share

5.5 4.5. Conclusión de la Fase 4

La Fase 4 culmina con la validación exitosa del procesador en hardware, demostrando que el diseño del microprocesador segmentado es capaz de ejecutar las instrucciones correctamente en un entorno real. La interacción con los periféricos y la ejecución adecuada de las aplicaciones confirmaron que el procesador está funcionando como se esperaba. Este paso demuestra la capacidad de transferir el diseño desde una simulación a un entorno físico, validando su funcionamiento en hardware y dejando el sistema listo para posibles ampliaciones o mejoras en las siguientes fases del proyecto.